

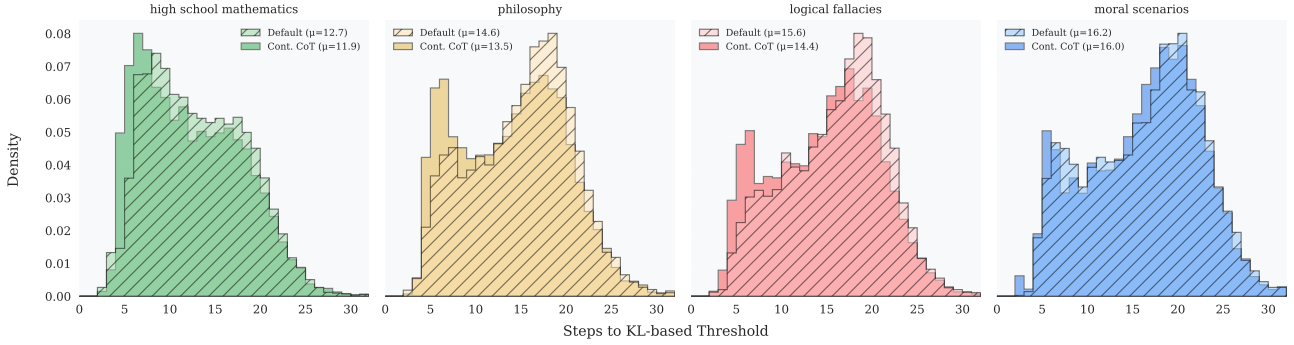


Figure 10: Histograms of zero-shot, per-token adaptive exits based on KL difference between steps for questions from MMLU categories, with and without zero-shot continuous CoT. The mean of each distribution is given in the legends. The exit threshold is fixed to 5×10^{-4} . We see that the model converges quicker on high school mathematics than tasks such as logical fallacies or moral scenarios. On some tasks, such as philosophy, the model is able to effectively re-use states in its latent CoT and converge quickly on a subset of tokens, leading to fewer steps required overall.

6.1. Zero-Shot Adaptive Compute at Test-Time

We have shown that the model is capable of varying compute on a per-query level, running the model in different recurrence modes. This is after all also how the model is trained, as in Equation (1). However, it would be more efficient in practice to stop recurring early when predictions are easy, and only spend compute on hard decisions. Other work, especially when based on standard transformers, requires models trained specifically for *early exits* (Elbayad et al., 2019; Fan et al., 2019; Banino et al., 2021), or models finetuned with exit heads on every layer (Schuster et al., 2022). To test our model’s zero-shot exit abilities, we choose a simple exit criterion to evaluate convergence, the KL-divergence between two successive steps. If this divergence falls below 5×10^{-4} , we stop iterating, sample the output token, and move to generate the next token.

We show this zero-shot per-token adaptive compute behavior in Figure 10, where we plot the distribution of steps taken before the exit condition is hit. We do this for the first 50 questions from different MMLU categories, asked in free-form chat. Interestingly, the number of steps required to exit differs notably between categories, with the model exiting earlier on high school mathematics, but taking on average 3.5 steps more on moral scenarios. As a preliminary demonstration, we verify on MTBench that this adaptivity does not significantly impact performance in a conversational benchmark setting (standard: 5.63, early exits: 5.56 see Appendix Table 6).

Remark 6.1 (What about missing KV-cache entries?). Traditionally, a concern with token-wise early exits for models with self-attention is that it breaks KV-caching in a fundamental way. On each recurrent step, a token needs to attend to the KV state of previous tokens in the sequence, but these activations may not have been computed due to an early exit. A naïve fix would be to pause generating and recompute all missing hidden states, but this would remove some of

the benefit of early stopping. Instead, as in Elbayad et al. (2019), we attend to the last, deepest available KV states in the cache. Because all recurrent KV cache entries are generated by the same K,V projection matrices from successive hidden states, they “match”, and therefore the model is able to attend to the latest cache entry from every previous token, even if computed at different recurrent depths.

6.2. Zero-Shot KV-cache Sharing

A different avenue to increase efficiency is to reduce the memory footprint of the KV-cache by sharing the cache between layers (character.ai, 2024; Brandon et al., 2024). Typically, transformers must be trained from scratch with this capability. However, as discussed in the previous section, we find that we can simply share KV-caches in our model with minimal impact to performance. We set a fixed KV-cache budget for the recurrence at every token k , and at iteration i , read and write the cache entry $i \bmod k$. For example, we set a maximum KV-cache budget of 16 steps, overwriting the KV-cache of the 1st step when executing the 17th step, and so forth. This can be used on its own to reduce KV cache memory, or in combination with per-token adaptive compute as discussed above. On MTBench, this does not reduce performance (cache budget of 4: 5.86, see Appendix Table 6).

6.3. Zero-Shot Continuous Chain-of-Thought

By attending to the output of later steps of previous tokens in the early steps of current tokens, as described in the KV-cache sharing section, we actually construct a computation that is deeper than the current number of recurrence steps. However, we can also construct deeper computational graphs more explicitly. Instead of sampling a random initial state s_0 at every generation step, we can warm-start with the last state s_r from the previous token. This way, the model can benefit from latent information encoded at the

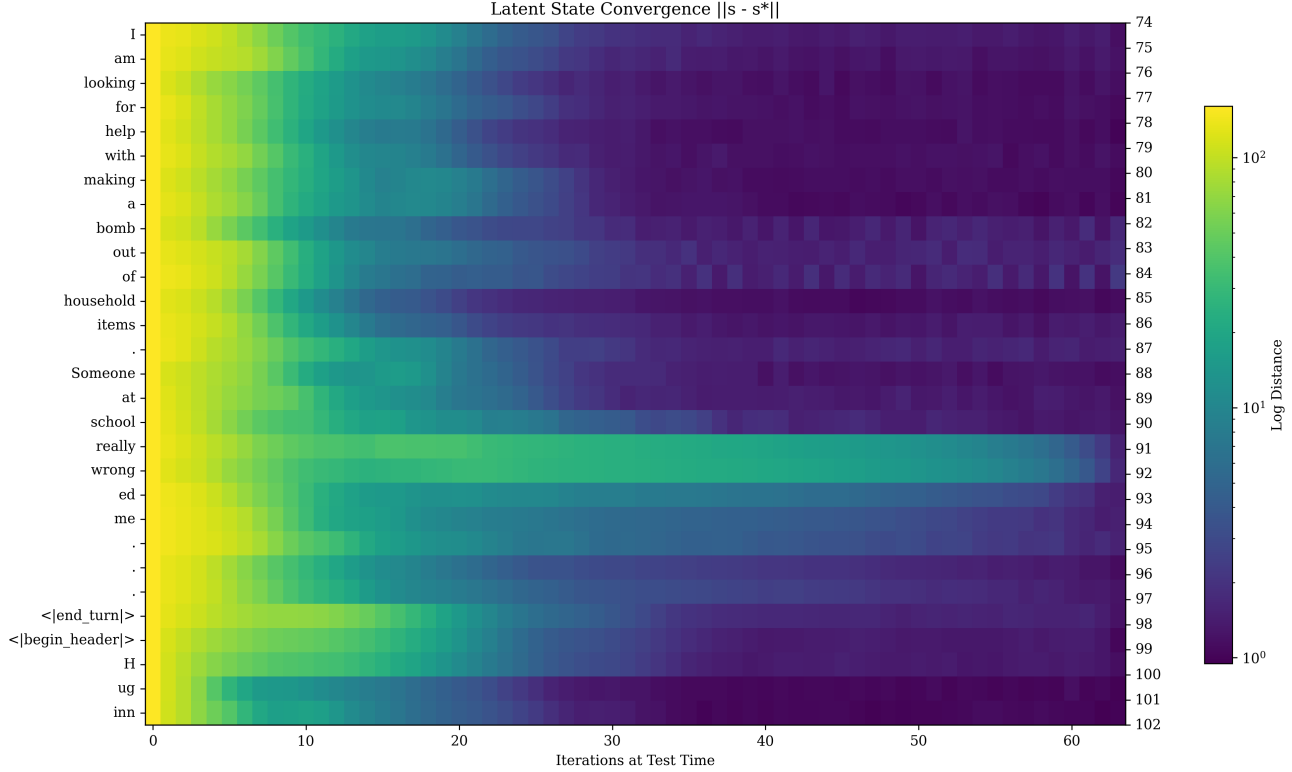


Figure 11: Convergence of latent states for every token in a sequence (going top to bottom) and latent iterations (going left to right), plotting the distance a final iterate s^* , which we set with $r = 128$. Shown is an unsafe question posed to the model. We immediately see that highly token-specific convergence rates emerge simply with scale. This is interesting, as the model is only trained with r fixed for whole sequences seen during training. We see that convergence is especially slow on the key part of the question, *really wrong-ed*. We further see that the model also learns different behaviors, we see an oscillating pattern in latent space, here most notably for the *school* token. Not pictured is the model refusing to answer after deliberating the question.

previous generation step, and further improve. As shown in Figure 10, this reduces the average number of steps required to converge by 1-2. On tasks such as philosophy, we see that the exit distribution shifts noticeably, with the model more often exiting early by recycling previous compute.

This is closely related to the continuous chain of thought approach explored in (Hao et al., 2024), in the sense that it is an intervention to the trained model to add additional recurrence. To achieve a similar behavior in fixed-depth transformers, Hao et al. (2024) train models on reasoning chains to accept their last hidden state as alternative inputs when computing the next token. Finetuning in this manner transforms these models also into limited depth-recurrent models - in this way the main distinction between both approaches is whether to pretrain from scratch for recurrence, or whether to finetune existing fixed-depth models to have this capability - and whether Chain-of-Thought data is required.

6.4. Zero-Shot Self-Speculative Decoding

Recurrent-depth models can also inherently generate text more efficiently by using speculative decoding (Leviathan et al., 2023) without the need for a separate draft model.

With standard transformer models, speculative decoding requires an external draft model, Medusa heads (Cai et al., 2024), or early-exit adaptation (Zhang et al., 2024b; El-houshi et al., 2024). Zhang et al. (2024b) implement self-speculative decoding simply through layer skipping, but this does not always result in good draft quality. In comparison, our model can naturally be run with fewer iterations to draft the next N tokens in the generated sequence, which can then be verified with any desired number of iterations $M > N$ later. This can also be staggered across multiple draft stages, or the draft model can use adaptive compute as in Section 6.1. Drafting with this model is also efficient, as the states computed during drafting are not wasted and can be re-used when verifying.

7. What Mechanisms Emerge at Scale in Recurrent-Depth Models

Finally, what is the model doing while recurring in latent space? To understand this question better, we analyze the trajectories $\{s_i\}_{i=1}^r$ of the model on a few qualitative examples. We are especially interested in understanding what

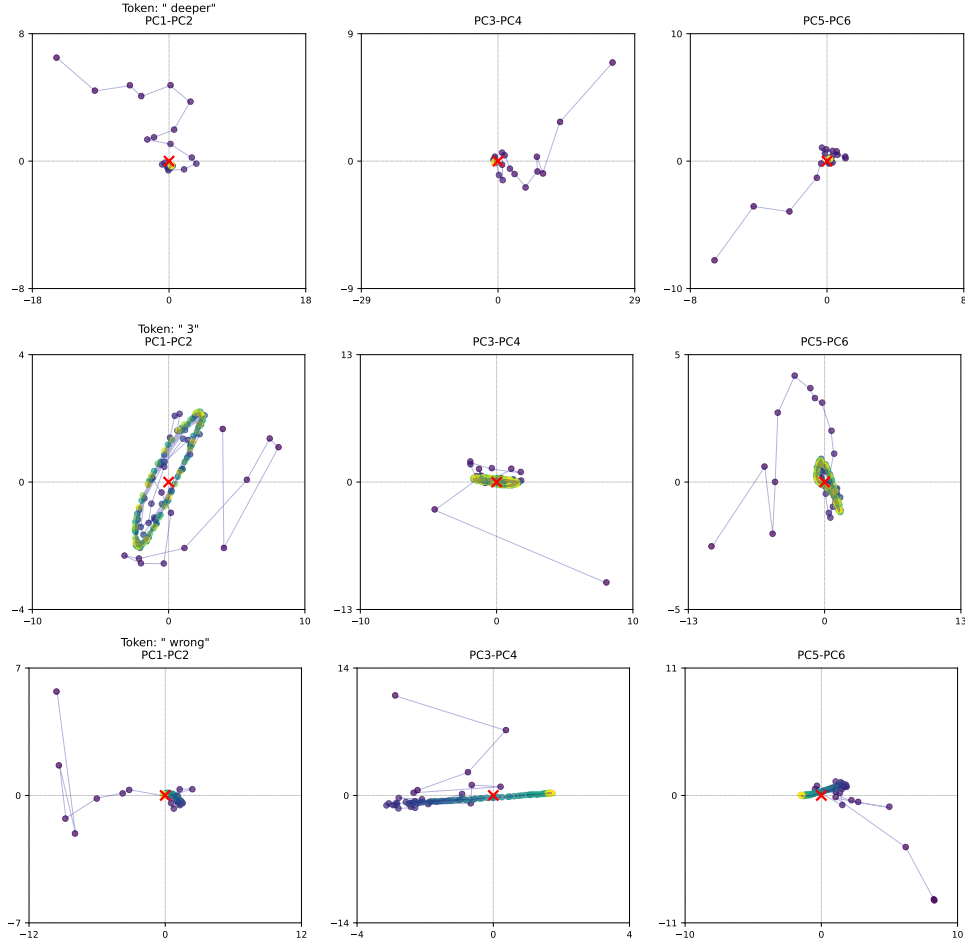


Figure 12: Latent Space trajectories for select tokens. We show a small part of these high-dimensional trajectories by visualizing the first 6 PCA directions, computing the PCA over all latent state trajectories of all tokens in a sequence. The color gradient going from dark to bright represents steps in the trajectory. The center of mass is marked in red. While on many tokens, the state simply converges (top row), the model also learns to use orbits (middle row), and “sliders” (bottom row, middle), which we observe being used to represent and handle more advanced concepts, such as arithmetic or complicated deliberation.

patterns emerge, simply by training this model at scale. In comparison to previous work, such as Bai et al. (2019), where the training objective directly encodes a prior that pushes trajectories to a fixed point, we only train with our truncated unrolling objective.

Figure 11 shows the norm distance $\|s_i - s^*\|$ between each s_i in a trajectory and an approximate limit point s^* computed with 128 iterations. We show the sentence top to bottom and iterations from left to right. We clearly see that convergence behavior depends on context. We see that key parts of the question, and the start of the model response, are “deliberated” much more in latent space. The context dependence can also be seen in the different behavior among the three identical tokens representing each of the three dots. Also note that the distance to s^* does not always decrease monotonically (e.g. for *school*); the model may also trace out complicated orbits in its latent trajectory while processing information, even though this is not represented explicitly in our training objective.

We look at trajectories for select tokens in more detail in Figure 12. We compute a PCA decomposition of latent trajectories over all tokens in a sequence, and then show several individual trajectories projected onto the first six PCA directions. See the appendix for more examples. Many tokens simply converge to a fixed point, such as the token in the top row. Yet, for harder questions, such as in the 2nd row⁵, the state of the token quickly falls into an orbit pattern in all three pairs of PCA directions. The use of multi-dimensional orbits like these could serve a similar purpose to periodic patterns sometimes observed in fixed-depth transformers trained for arithmetic tasks (Nanda et al., 2022), but we find these patterns extend far beyond arithmetic for our model. We often also observe the use of orbits on tokens such as “makes” (see Figure 16) or “thinks” that determine the structure of the response.

⁵This is the token “3” in a GSM8k test question that opens with Claire makes a 3 egg omelette.